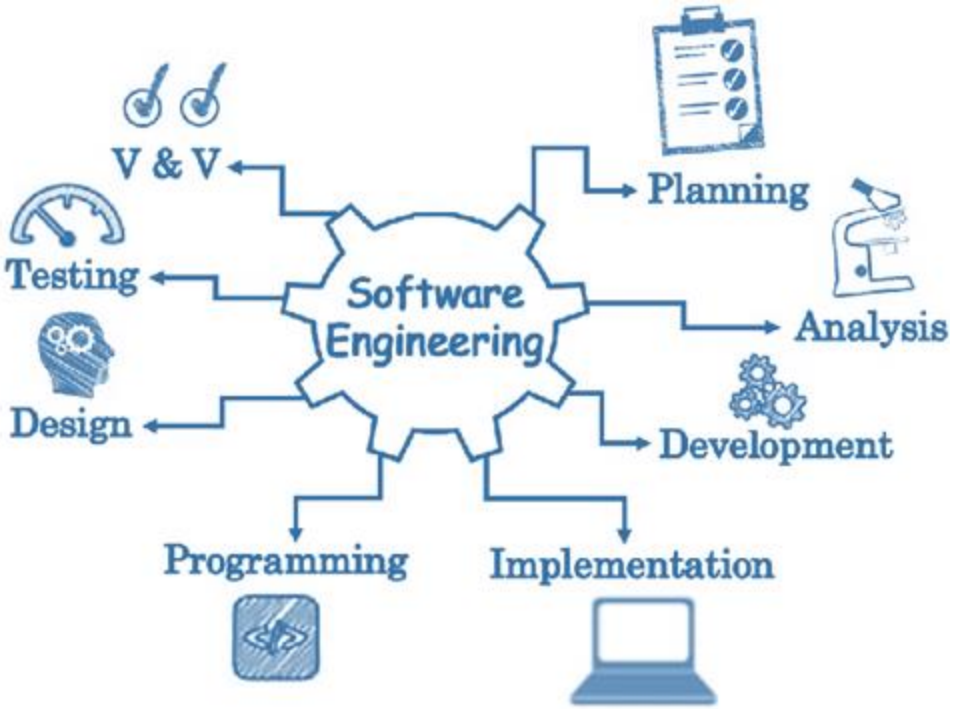
Software Engineering (CMP 519.3)



- Instructor
- Birodh Rijal

birodh.rijal@eemc.edu.np

Unit VI: Software Quality Assurance and Testing (5 Hours)

6.1 Software Quality Concepts and Attributes

- Reliability, Usability, Efficiency, Maintainability
- Software Safety and Security

6.2 Software Cost Estimation

- Introduction to the COCOMO Model
- Estimation Techniques and Tools

6.3 Software Quality Assurance Planning and Process

- Quality Assurance Activities
- Role of QA in the Software Lifecycle

6.4 Testing Concepts

- Faults, Erroneous States, and Failures
- Test Cases, Test Stubs, and Drivers

6.5 Testing Activities

- Unit Testing
- Integration Testing
- System Testing
- Regression Testing

6.6 Automated Testing Frameworks

- Introduction to Tools like Selenium and JUnit
- Integrating Testing Tools with CI/CD Pipelines

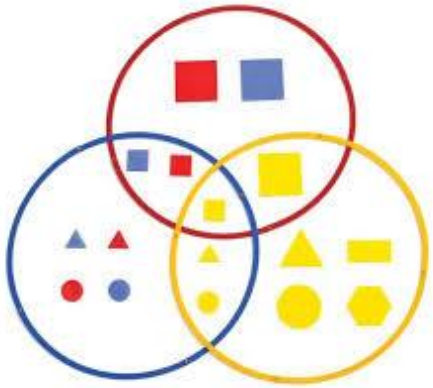
Software Quality Concepts and Attributes

- Software Quality Concepts and Attributes

Concepts

Software Quality Concepts refers to the general ideas or principles that ensure software is good.





Attributes

Attributes are the specific characteristics or features that contribute to that quality.

Software Quality Concepts and Attributes

- Software Quality Concepts and Attributes

  - Software quality encompasses the characteristics that determine the value of software to its users.

  - It is defined by how well the software meets specified requirements, user expectations, and industry standards.



Software Quality Concepts and Attributes

- Software Quality Concepts and Attributes

Key Concepts of Software Quality:

1.Conformance to Requirements – Software should fulfill functional and non-functional requirements.

2.Reliability – The system should perform consistently under expected conditions.

3.Maintainability – Easy to modify, update, and fix issues.

4.Usability – The software should be user-friendly and intuitive.

5.Efficiency – Optimal resource utilization with minimal overhead.

Software Quality Concepts and Attributes

- Software Quality Attributes

Reliability

•**Definition** : The ability of the software to perform consistently under specified conditions.

•**Sub-Attributes** :

- **Maturity** : Low frequency of failures.
- **Availability** : The software is accessible when needed.
- **Fault Tolerance** : Continues to operate despite errors.
- **Recoverability** : Restores data and functionality after a failure.

Usability

•**Definition** : The ease with which users can operate and understand the software.

•**Sub-Attributes** :

- **Appropriateness Recognizability** : Users can recognize if the software is suitable for their needs.
- **Learnability** : Users can easily learn how to use the software.
- **Operability** : The software is easy to control and use.
- **User Error Protection** : Prevents users from making mistakes.

Software Quality Concepts and Attributes

- Software Quality Attributes

Performance Efficiency

•**Definition** : The performance of the software relative to the amount of resources used.

•**Sub-Attributes** :

- **Time Behavior** : Response and processing times.
- **Resource Utilization** : Efficient use of CPU, memory, etc.
- **Capacity** : Ability to handle maximum workloads.

Maintainability

•**Definition** : The ease with which the software can be modified to correct defects or improve performance.

•**Sub-Attributes** :

- **Modularity** : Components are independent and reusable.
- **Reusability** : Code can be reused in other projects.
- **Analyzability** : Easy to diagnose issues.
- **Modifiability** : Changes can be implemented without side effects.

Software Quality Concepts and Attributes

- Software Quality Attributes

Compatibility

•**Definition** : The ability of the software to interact with other systems or components.

•**Sub-Attributes** :

- **Interoperability** : Works with other systems.
- **Co-existence** : Operates alongside other software without conflict.

Security

•**Definition** : The ability of the software to protect data and maintain integrity.

•**Sub-Attributes** :

- **Confidentiality** : Ensures sensitive data is protected.
- **Integrity** : Prevents unauthorized modification of data.
- **Non-repudiation** : Ensures actions cannot be denied.
- **Accountability** : Tracks user actions.

Software Quality Concepts and Attributes

- Software Quality Attributes

Portability

•**Definition** : The ease with which the software can be transferred to different environments.

•**Sub-Attributes** :

- **Adaptability** : Can be adapted to new environments.
- **Installability** : Easy to install and configure.
- **Replaceability** : Can replace another software component.

Software Quality Concepts and Attributes

- Software Quality Attributes

Standards

1.ISO 9001:2015 :

1. A general standard for Quality Management Systems (QMS).
2. Focuses on improving processes, meeting customer needs, and ensuring compliance across industries.

2.ISO 25002:2024 :

1. A specialized standard for data quality.
2. Provides a framework to evaluate and improve the quality of data used in systems, ensuring it is accurate, reliable, and usable.

Software Quality Concepts and Attributes

- Software Safety and Security

Software safety ensures that a system operates without causing harm to people, property, or the environment, even in the presence of **failures or unexpected conditions**. It focuses on **preventing accidents** caused by software malfunctions.

Software Security



Software security protects systems, data, and users from **malicious attacks, unauthorized access, or breaches**. It focuses on **confidentiality, integrity, and availability** (CIA triad) of information and services.

Software Quality Concepts and Attributes

- Security Life-cycle models

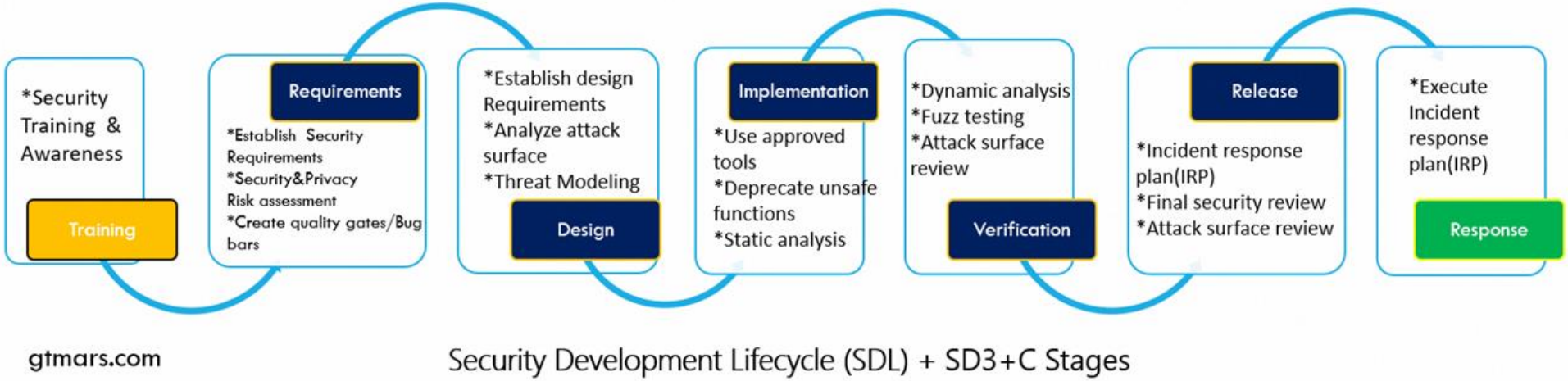
Microsoft defined a collection of principles it calls

- ***Secure by Design,***
- ***Secure by Default,***
- ***Secure in Deployment, and Communications (SD3+C)***

to help determine where security efforts are needed.

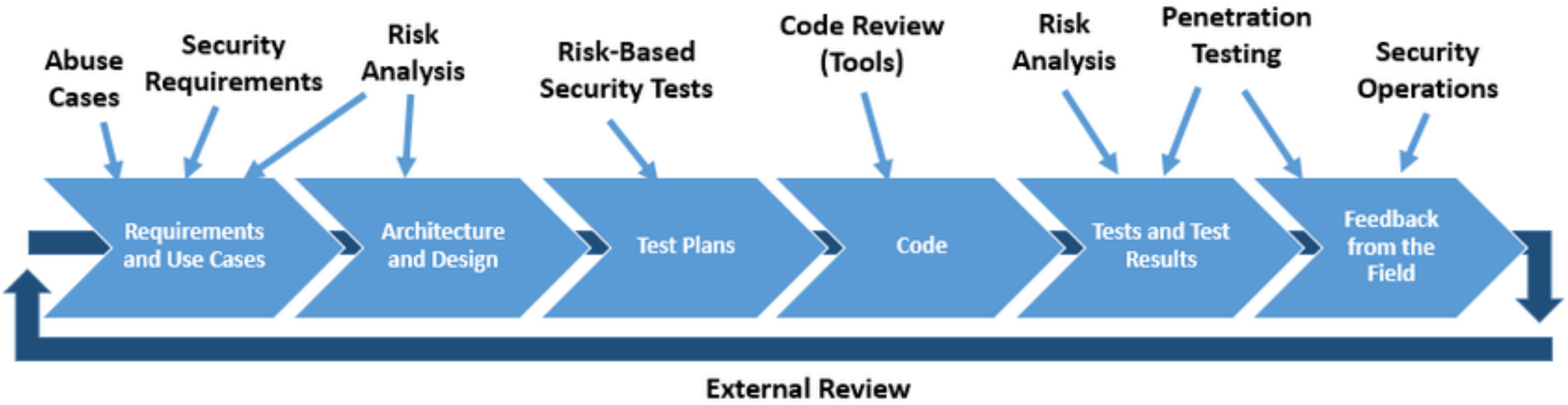
Software Quality Concepts and Attributes

- Secure Software Development Process Model at Microsoft



Software Quality Concepts and Attributes

- Software Security touchpoints



Software Quality Concepts and Attributes

- Security Requirements Engineering

Table 1. SQUARE steps

Step 1: Agree on definitions
Input: Candidate definitions from IEEE and other standards **Technique:** Structured interviews, focus group **Participant:** Stakeholders, requirements team **Output:** Agreed-to definitions
Step 2: Identify assets and security goals
Input: Definitions, candidate goals, business drivers, policies and procedures, examples **Technique:** Facilitated work session, surveys, interviews **Participant:** Stakeholders, requirements engineer **Output:** Assets and goals
Step 3: Develop artifacts to support security requirements definition
Input: Potential artifacts (e.g., scenarios, misuse cases, templates, forms) **Technique:** Work session **Participant:** Requirements engineer **Output:** Needed artifacts: scenarios, misuse cases, models, templates, forms

Software Quality Concepts and Attributes

- Security Requirements Engineering

Step 4: Perform risk assessment
Input: Misuse cases, scenarios, security **Technique:** Risk assessment method, analysis of anticipated risk against organizational risk tolerance, including threat analysis **Participant:** Requirements engineer, risk expert, stakeholders **Output:** Risk assessment results
Step 5: Select elicitation techniques
Input: Goals, definitions, candidate techniques, expertise of stakeholders, organizational style, culture, level of security needed, cost/benefit analysis, etc. **Technique:** Work session **Participant:** Requirements engineer **Output:** Selected elicitation techniques
Step 6: Elicit security requirements
Input: Artifacts, risk assessment results, selected techniques **Technique:** Joint Application Development (JAD), interviews, surveys, model-based analysis, checklists, lists of reusable requirements types, document reviews **Participant:** Stakeholders facilitated by requirements engineer **Output:** Initial cut at security requirements

Software Quality Assurance Planning and Process

- SQA Plan

A standard for SQA plans has been published by the IEEE [IEE17]. The standard recommends a structure that identifies:

1. The purpose and scope of the plan,
2. A description of all software engineering work products (e.g., models, documents, source code) that fall within the purview of SQA,
3. All applicable standards and practices that are applied during the software process,
4. SQA actions and tasks (including reviews and audits) and their placement throughout the software process,
5. The tools and methods that support SQA actions and tasks,
6. Software configuration management (procedures,
7. Methods for assembling, safeguarding, and maintaining all SQA-related records, and
8. Organizational roles and responsibilities relative to product quality”

Software Quality Assurance Planning and Process

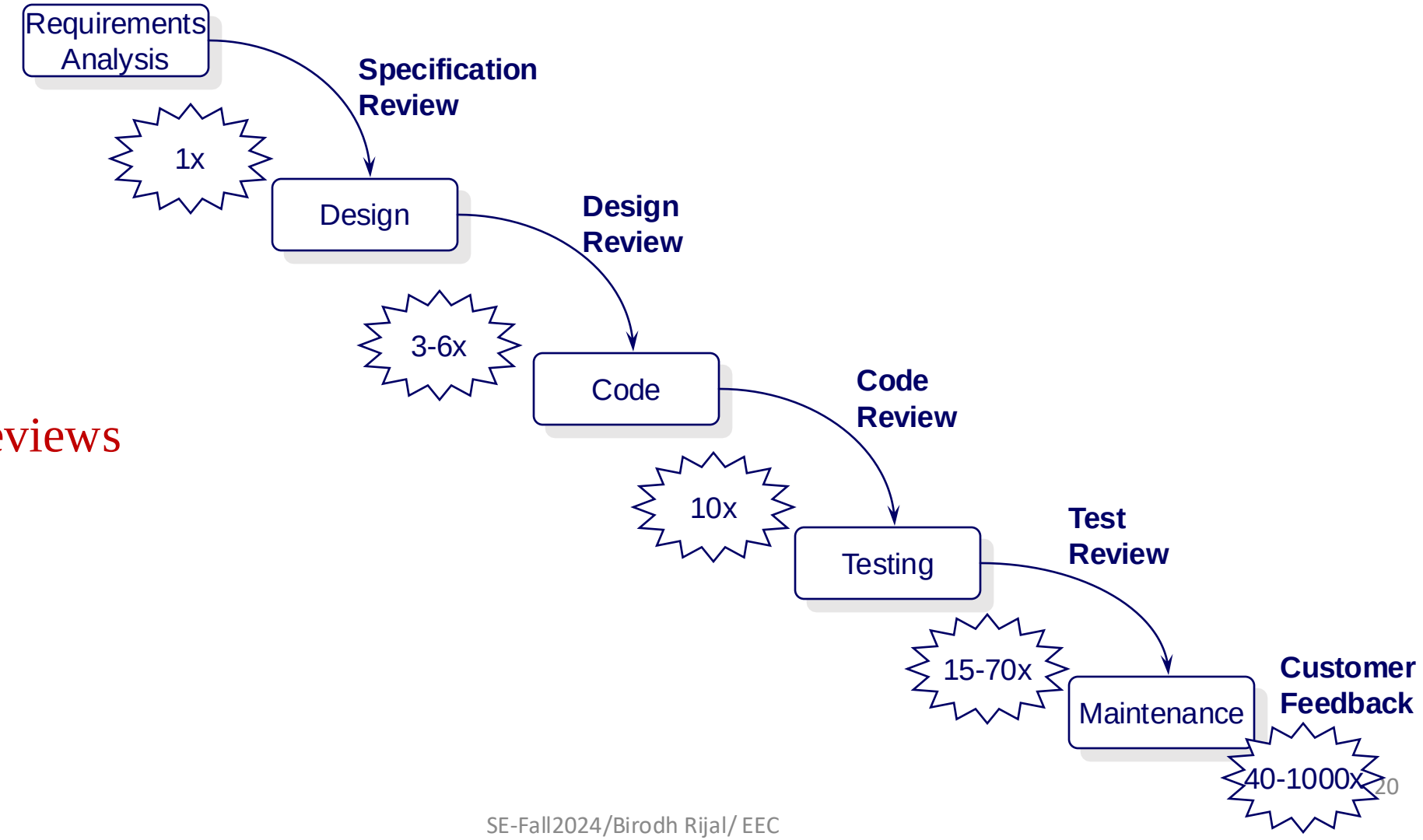
- SQA Activities
  - Applying technical methods
    - To help the analyst achieve a high quality specification and a high quality design
  - Conducting formal technical reviews
    - A stylized meeting conducted by technical staff with the sole purpose of uncovering quality problems
  - Testing Software
    - A series of test case design methods that help ensure effective error detection
  - Enforcing standards
  - Controlling change
    - Applied during software development and maintenance
  - Measurement
    - Track software quality and asses the ability of methodological and procedural changes to improve software quality
  - Record keeping and reporting
    - Provide procedures for the collection and dissemination of SQA information

Software Quality Assurance Planning and Process

- Quality Reviews
  - The fundamental method of validating the quality of a product or a process.
  - Applied during and/or at the end of each life cycle phase
    - Point out needed improvements in the product of a single person or team
    - Confirm those parts of a product in which improvement is either not desired or not needed
    - Achieve technical work of more uniform, or at least more predictable, quality than what can be achieved without reviews, in order to make technical work more manageable
  - Quality reviews can have different intents:
    - review for defect removal
    - review for progress assessment
    - review for consistency and conformance

Software Quality Assurance Planning and Process

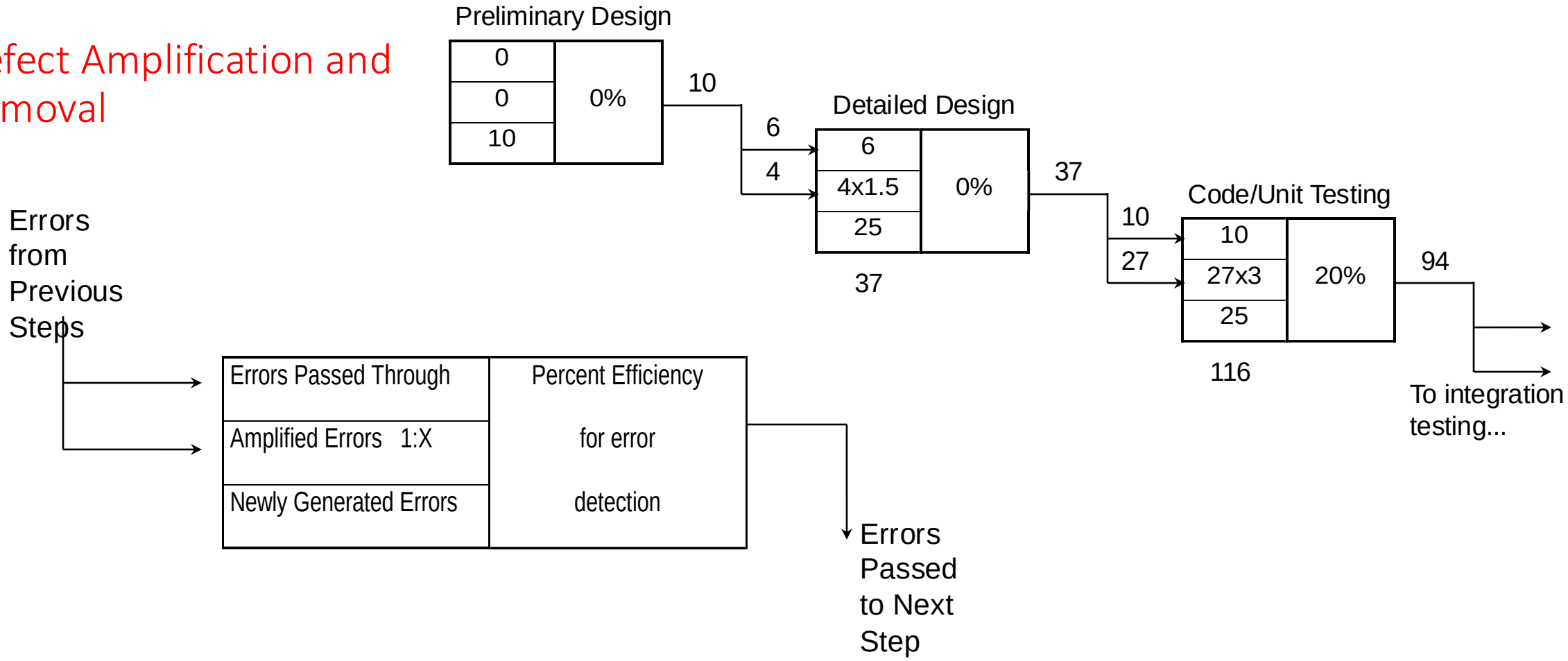
- Quality Reviews



[Adapted from Pressman 4th Ed]

Software Quality Assurance Planning and Process

Defect Amplification and Removal



[Adapted from Pressman 4th Ed]

Testing Concept

Faults (Defects or Bugs)

- A **fault** is an incorrect step, process, or data definition in a software program.
- It is a static defect in the software code or design.
- Faults occur due to programming errors, design mistakes, or incorrect system configurations.
    - **Example:** A developer mistakenly writes x = y + 1; instead of x = y - 1;.

Erroneous States (Errors)

- An **erroneous state** (or simply an "error") occurs when a fault gets triggered, leading to an incorrect internal system state.
- This is a **dynamic** issue that happens during execution.
- Not all erroneous states lead to failures, as some may be handled internally by the system.
    - **Example:** Due to the above fault, a program might compute an incorrect variable value during execution.

Failures

- A **failure** occurs when an erroneous state propagates to the system's output, causing it to behave incorrectly.
- This is the observable malfunction that users experience.
    - **Example:** If an erroneous state causes a banking system to miscalculate a customer's balance, leading to an incorrect transaction, it results in a failure.

[Adapted from Pressman 4th Ed]

Testing Concept

Test Case

- A **Test Case** is a set of conditions or variables that a tester uses to determine whether a system or application behaves as expected under specific circumstances.
- It defines the steps to be executed, the input data to be used, and the expected results, to verify that the system functions correctly.
- A test case is a document which has a set of conditions/ actions/ steps that are performed on the software application in order to verify the expected functionality of the feature under test.
- A test case is mostly used to validate whether the developed feature is working as per requirement or not.

Test Case ID	Test Case Description	Pre-conditions	Steps	Expected Result	Actual Result	Status
CUS_001	Verify products can be added to cart	Login to Customer App	1. From the home page add 2 products to cart. 2. Click on the cart icon.	1. “Add to cart” button text should be changed to “Added to cart”. 2. Cart should have 2 products added.	1. “Add to cart” button text changed to “Added to cart”. 2. Added 2 products to cart.	Pass
CUS_002	Verify products can be checkout	1. Login to Customer App 2. Add products to cart.	1. Click on the cart icon. 2. Click on checkout button	1. Cart page should open. 2. Products should be checked out and the checkout page should open.	1. Cart page is opened. 2. Products should be checked out and the checkout page is opened.	Pass
CUS_003	Verify products can be ordered	1. Login to Customer App 2. Add products to cart and checkout.	1. Click on place order button	1. Add shipping address page should open.	1. The shipping address page should be opened.	Pass

[Adapted from Pressman 4th Ed]

Testing Concept

Test Stub

A **test stub** is a temporary, simplified version of a component that simulates the behavior of a module that is not yet developed.

- Used in **top-down integration testing** (when high-level modules are tested first).
- Returns **hardcoded or simulated responses** instead of real execution.
- Helps isolate and test the dependent module without waiting for the full system.

*Example: Imagine you are testing a **payment processing system**, but the actual **bank API is not ready**. You can create a stub that always returns **"Payment Successful"** instead of making real transactions.*

[Adapted from Pressman 4th Ed]

Testing Concept

Test Driver

A **test driver** is a temporary program used to simulate a calling module that **invokes** a component being tested.

- Used in **bottom-up integration testing** (when lower-level modules are tested first).
- Acts as a **placeholder for higher-level components** that are not yet implemented.
- Helps **trigger** and test individual components independently.

> *Imagine testing a **database module**, but the actual **front-end UI** isn't ready. A **test driver** can simulate user inputs and invoke the database functions for testing.*

[Adapted from Pressman 4th Ed]

Testing Activities

Unit Testing

- Unit testing is a **software testing technique** where individual components or functions of a program are tested in **isolation** to verify that they work correctly.
- It is typically performed by developers during the **development phase** to catch defects early.

    - **Tests Individual Units** – Focuses on functions, methods, or modules.
    - **Runs in Isolation** – Does not depend on other system components.
    - **Automated** – Often implemented using testing frameworks like **JUnit (Java), PyTest (Python), NUnit (.NET), Jest (JavaScript)**.
    - **Fast and Frequent** – Runs quickly and frequently during development.
      Improves Code Quality – Helps ensure code reliability and maintainability.

[Adapted from Pressman 4th Ed]

Testing Activities: Unit Testing

- Static Unit Testing
  - Code is examined over all possible behaviors that might arise during run time
  - Code of each unit is validated against requirements of the unit by reviewing the code
- Dynamic Unit Testing
  - A program unit is actually executed and its outcomes are observed
  - One observe some representative program behavior, and reach conclusion about the quality of the system
- Static unit testing is not an alternative to dynamic unit testing
- Static and Dynamic analysis are complementary in nature
- In practice, partial dynamic unit testing is performed concurrently with static unit testing
- It is recommended that static unit testing be performed prior to the dynamic unit testing

[Adapted from Pressman 4th Ed]

Testing Activities: Static Unit Testing

- In static unit testing code is reviewed by applying techniques:
  - **Inspection:** It is a step by step peer group review of a work product, with each step checked against pre-determined criteria
  - **Walkthrough:** It is review where the author leads the team through a manual or simulated executed of the product using pre-defined scenarios

•The idea here is to examine source code in detail in a systematic manner

•The objective of code review is to *review* the code, and *not* to evaluate the author of the code

•Code review must be planned and managed in a professional manner

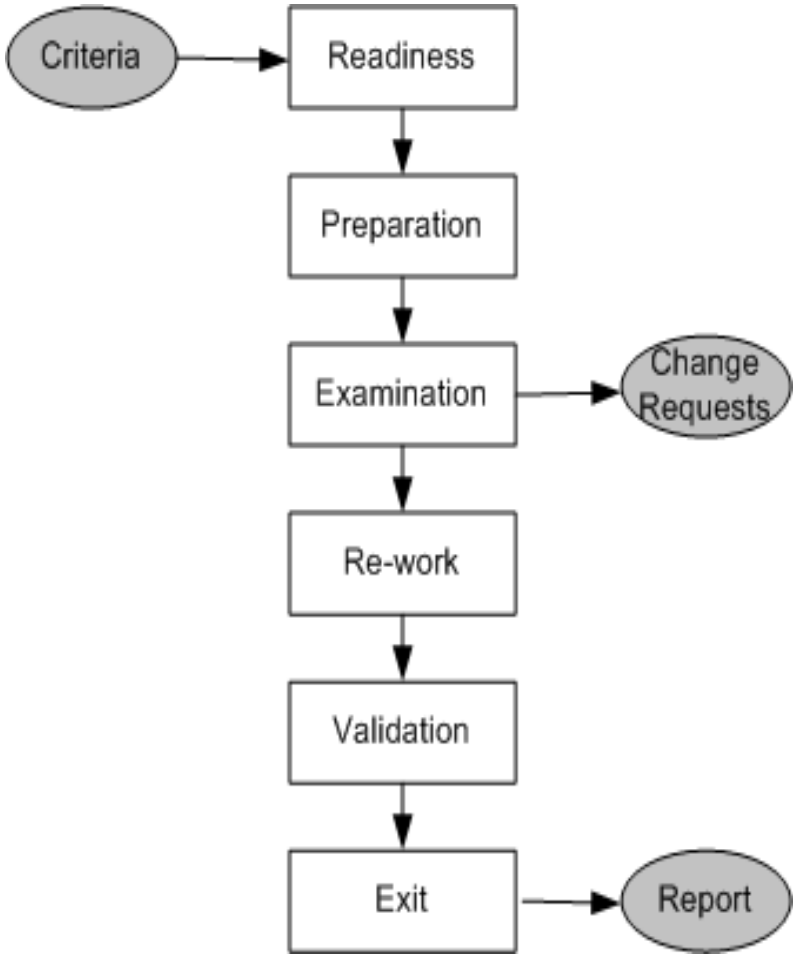
•The key to the success of code is to divide and conquer

- An examiner inspect small parts of the unit in isolation
  - nothing is overlooked
  - the correctness of all examined parts of the module implies the correctness of the whole module

[Adapted from Pressman 4th Ed]

Testing Activities: Static Unit Testing

- Step 1: **Readiness**
  - **Criteria**
    - **Completeness**
    - **Minimal functionality**
    - **Readability**
    - **Complexity**
    - **Requirements and design documents**
  - **Roles**
    - **Moderator**
    - **Author**
    - **Presenter**
    - **Record keeper**
    - **Reviewers**
    - **Observer**
- Step 2: **Preparation**
  - **List of questions**
  - **Potential Change Request (CR)**
  - **Suggested improvement opportunities**



[Adapted from Pressman 4th Ed]

Testing Activities: Static Unit Testing

- Step 3: **Examination**
  - The author makes a presentation
  - The presenter reads the code
  - The record keeper documents the CR
  - Moderator ensures the review is on track
- Step 4: **Re-work**
  - Make the list of all the CRs
  - Make a list of improvements
  - Record the minutes meeting
  - Author works on the CRs to fix the issue
- Step 5: **Validation**
  - CRs are independently validated
- Step 6: **Exit**
  - A summary report of the meeting minutes is distributes

A **Change Request (CR)** includes the following details:

- Give a brief description of the issue
- Assign a priority level (major or minor) to a **CR**
- Assign a person to follow it up
- Set a deadline for addressing a **CR**

[Adapted from Pressman 4th Ed]

Testing Activities: Dynamic **Unit Testing**

- The environment of a unit is emulated and tested in isolation
- The caller unit is known as *test driver*
  - A *test driver* is a program that invokes the unit under test (UUT)
  - It provides input data to unit under test and report the test result
- The emulation of the units called by the UUT are called *stubs*
  - It is a dummy program
- The *test driver* and the *stubs* are together called *scaffolding*
- The low-level design document provides guidance for selection of input test data

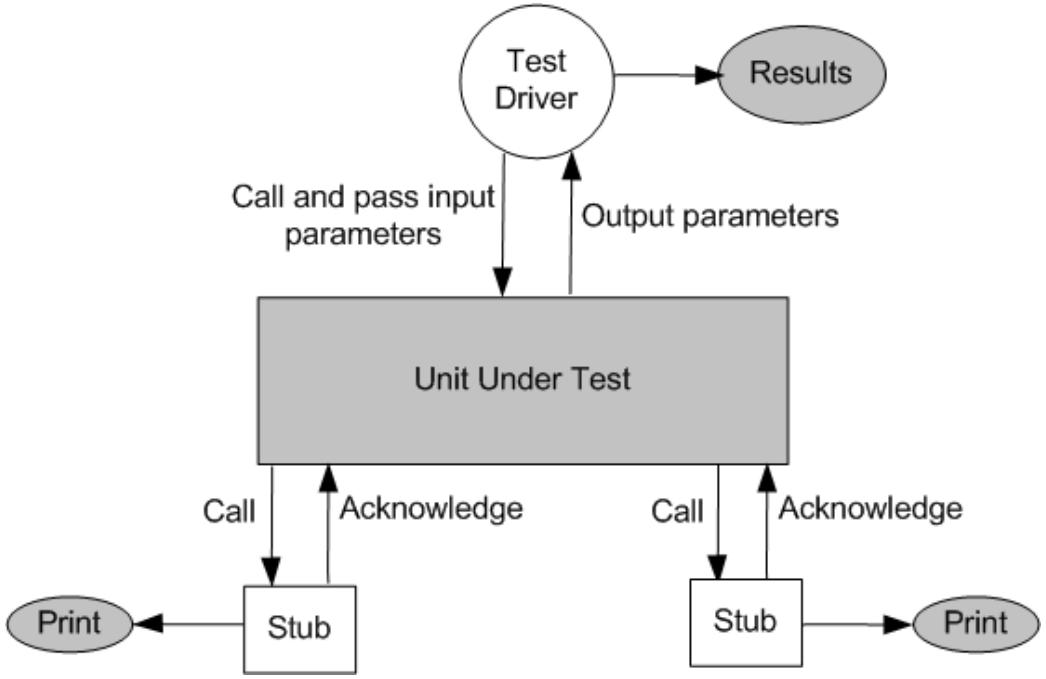


Figure : Dynamic unit test environment

[Adapted from Pressman 4th Ed]

Testing Activities: Dynamic **Unit Testing**

Selection of test data is broadly based on the following techniques:

- Control flow testing
  - Draw a control flow graph (CFG) from a program unit
  - Select a few control flow testing criteria
  - Identify a path in the CFG to satisfy the selection criteria
  - Derive the path predicate expression from the selection paths
  - By solving the path predicate expression for a path, one can generate the data
- Data flow testing
  - Draw a data flow graph (DFG) from a program unit and then follow the procedure described in control flow testing.
- Domain testing
  - Domain errors are defined and then test data are selected to catch those faults
- Functional program testing
  - Input/output domains are defined to compute the input values that will cause the unit to produce expected output values

[Adapted from Pressman 4th Ed]

Testing Activities: Dynamic **Unit Testing**

Junit Test Platform

- The **JUnit Platform** serves as a foundation for launching testing frameworks on the JVM.
- It also defines the TestEngine API for developing a testing framework that runs on the platform.
- Furthermore, the platform provides a Console Launcher to launch the platform from the command line and the JUnit Platform Suite Engine for running a custom test suite using one or more test engines on the platform.
- First-class support for the JUnit Platform also exists in popular IDEs (IntelliJ IDEA, Eclipse, NetBeans, and Visual Studio Code) and build tools (see Gradle, Maven, and Ant).

- Find more details here: https://junit.org/junit5/docs/current/user-guide/

[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

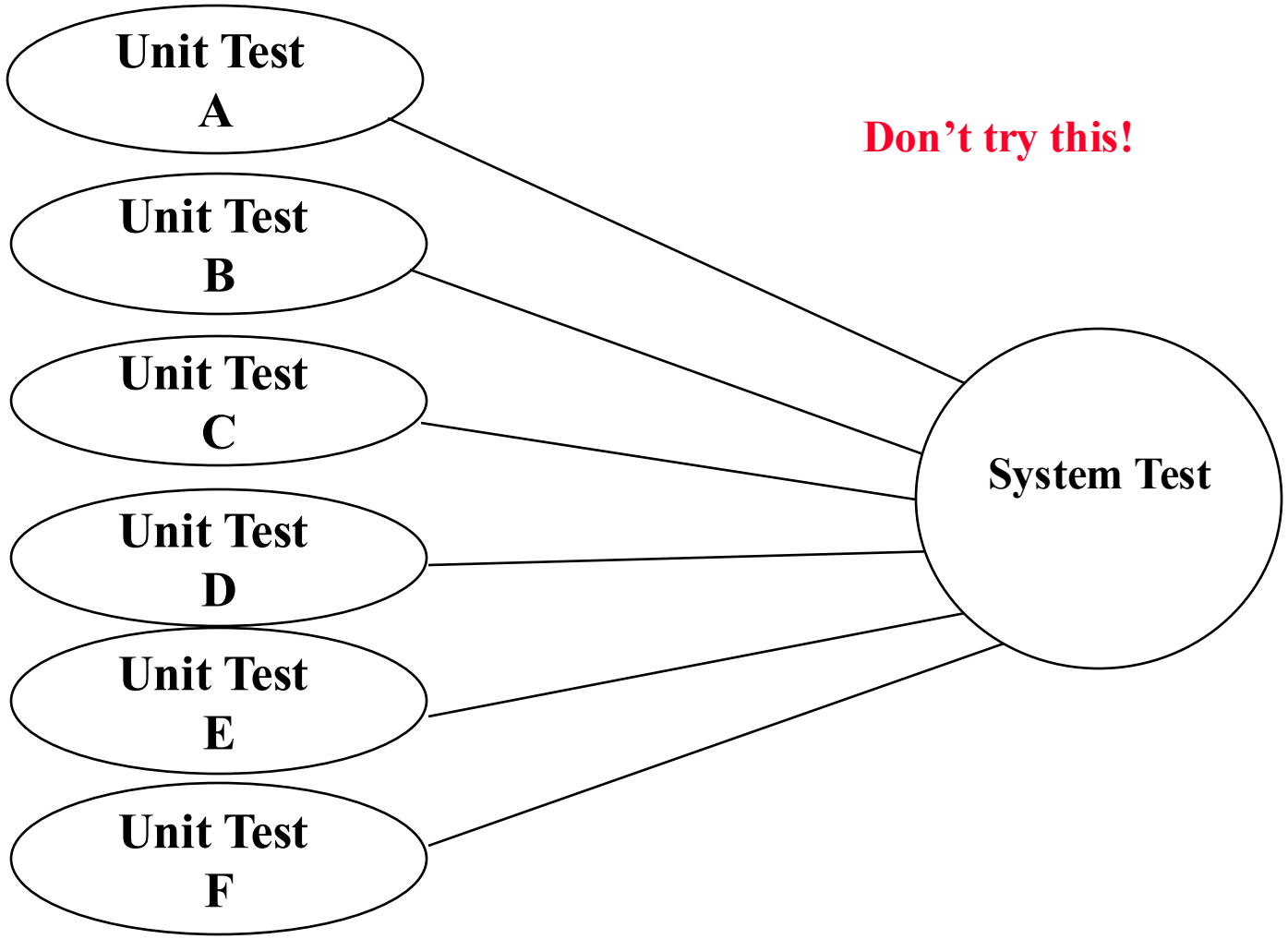
Integration Testing Strategy

- The entire system is viewed as a collection of subsystems (sets of classes) determined during the system and object design.
- The order in which the subsystems are selected for testing and integration determines the testing strategy
  - *Big bang integration (Nonincremental)*
  - *Bottom up integration*
  - *Top down integration*
  - *Sandwich testing*
  - *Variations of the above*
- For the selection use the system decomposition from the System Design

[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

Integration Testing:
Big-Bang Approach

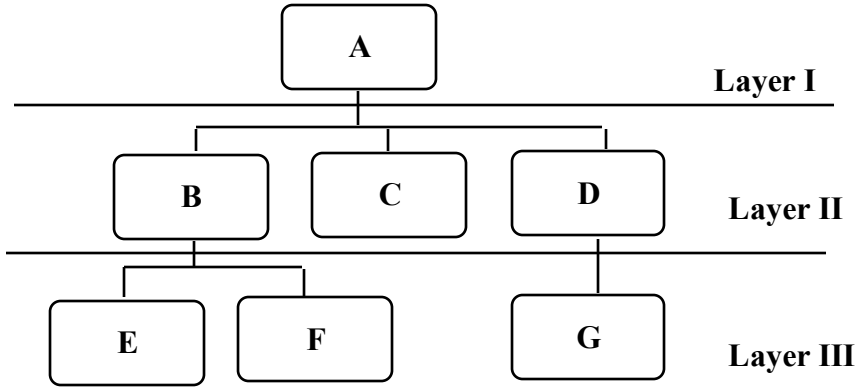


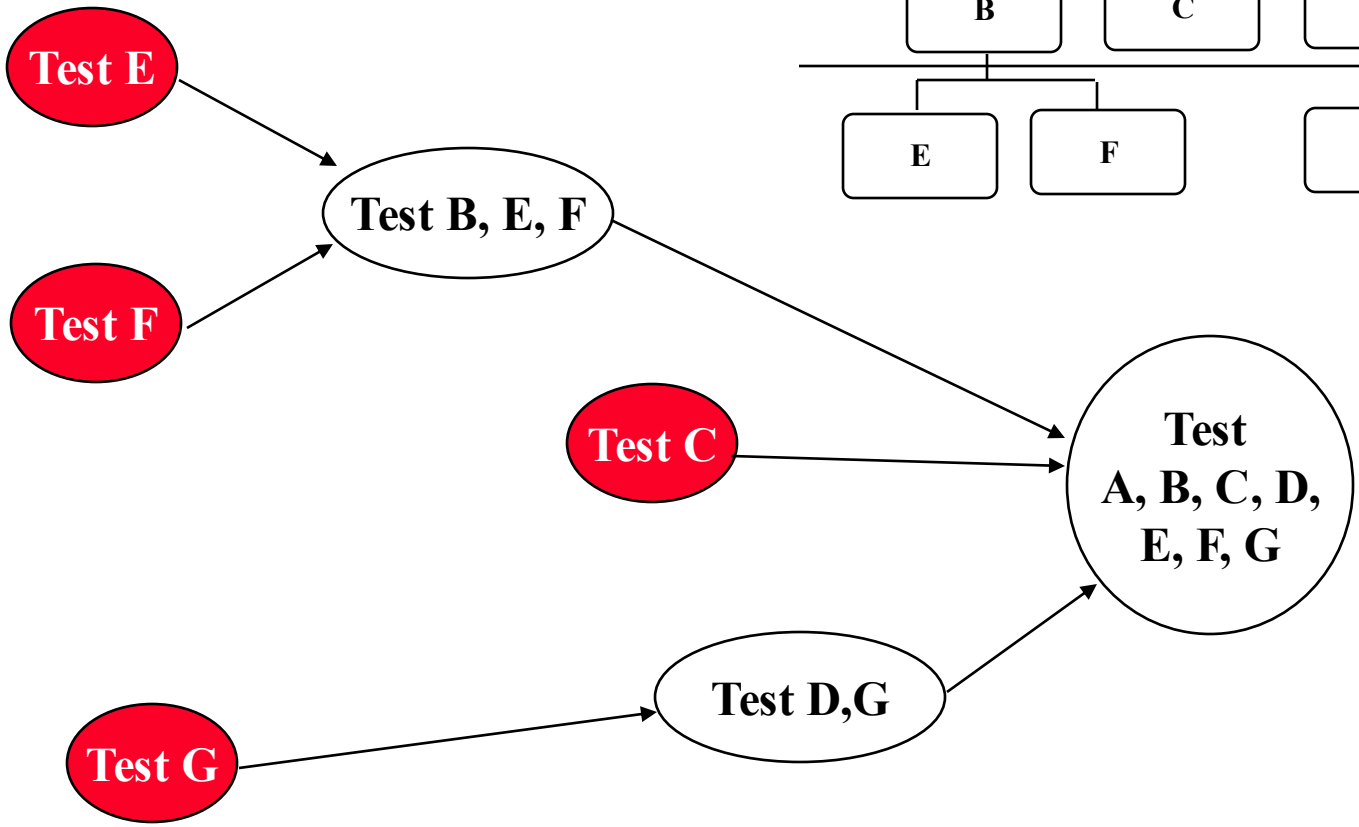
Don't try this!

[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

Integration Testing: Bottom-Up Integration





Special program needed to do the testing, Test Driver:

A routine that calls a subsystem and passes a test case to it

[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

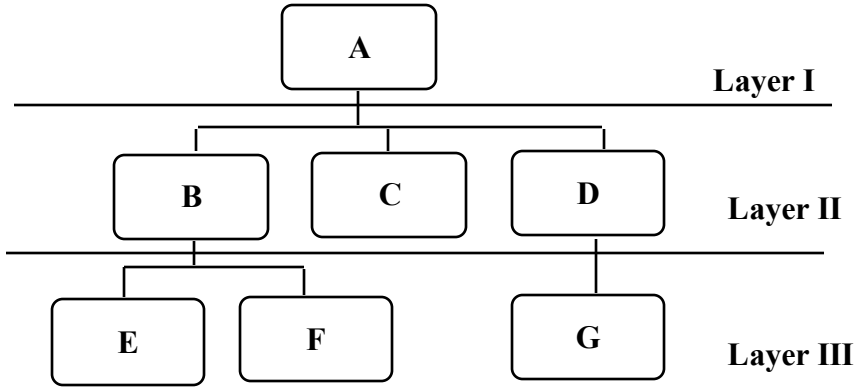
Integration Testing:
Bottom-Up Integration

- Bad for functionally decomposed systems:
  - Tests the most important subsystem (UI) last
- Useful for integrating the following systems
  - Object-oriented systems
  - real-time systems
  - systems with strict performance requirements

[Adapted from Pressman 4th Ed]

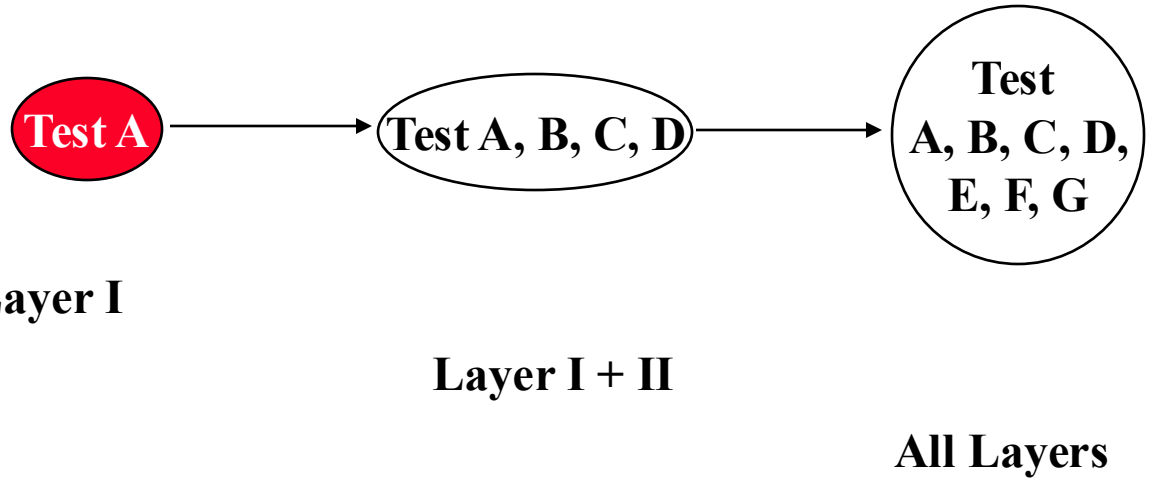
Testing Activities: Integration Testing

Integration Testing:
Top-Down Integration



Special program is needed to do the testing, *Test stub :*

A program or a method that simulates the activity of a missing subsystem by answering to the calling sequence of the calling subsystem and returning back fake data.



[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

Integration Testing: Pros and Cons of Top-Down Integration

- Test cases can be defined in terms of the functionality of the system (functional requirements)
- *Writing stubs can be difficult*: Stubs must allow all possible conditions to be tested.
- Possibly a very large number of stubs may be required, especially if the lowest level of the system contains many methods.
- One solution to avoid too many stubs: *Modified top-down testing strategy*
  - Test each layer of the system decomposition individually before merging the layers
  - Disadvantage of modified top-down testing: Both, stubs and drivers are needed

[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

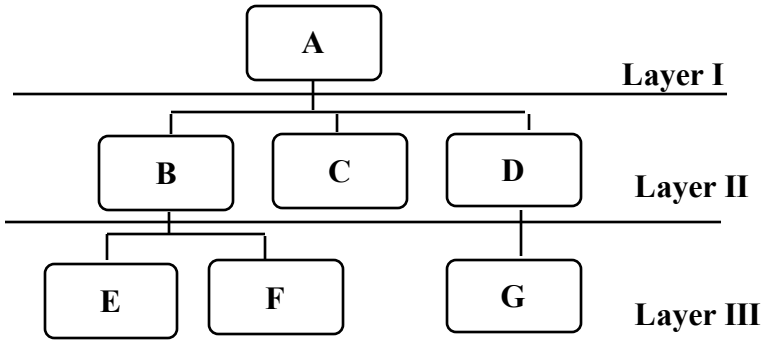
Integration Testing: Sandwich Integration

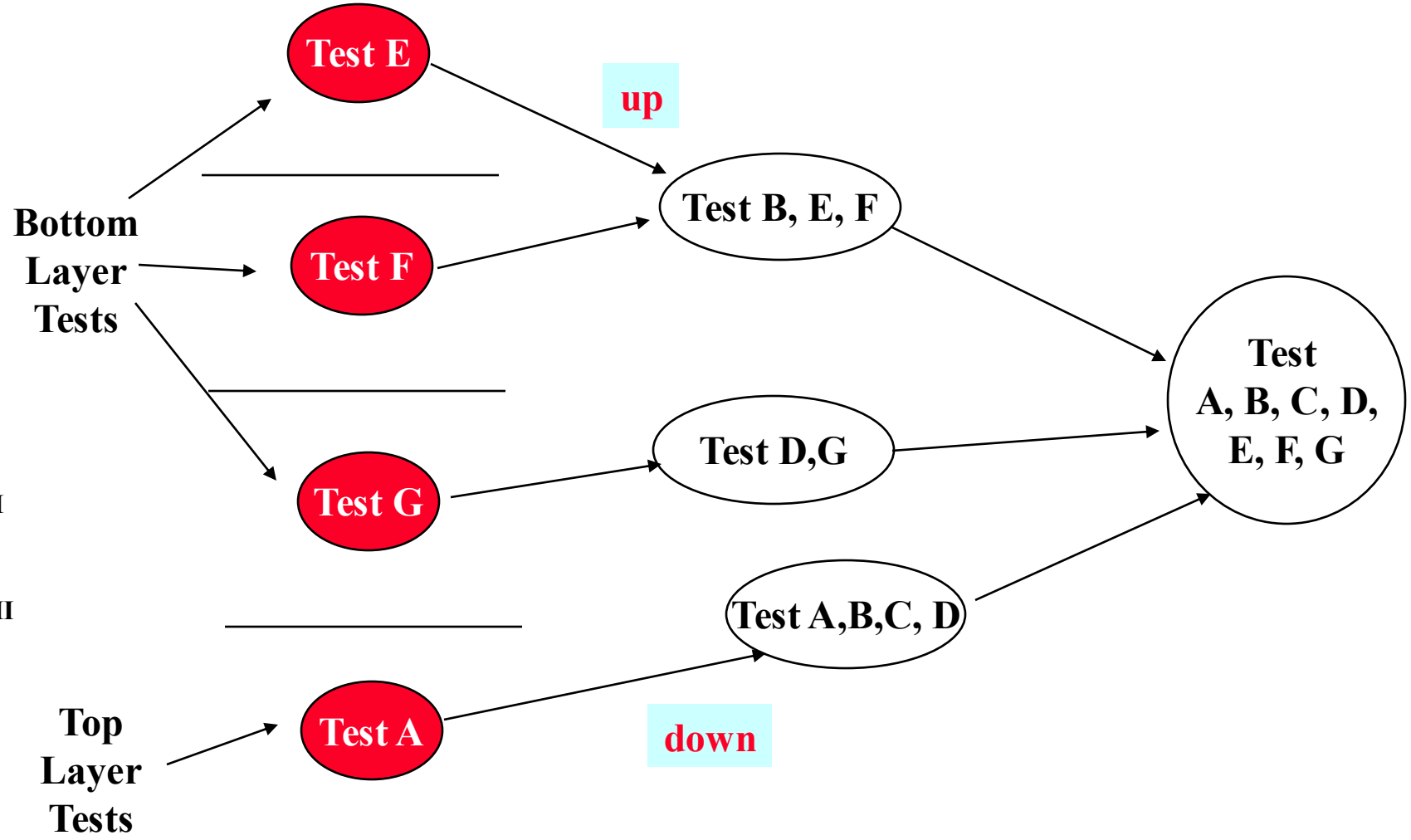
- Combines top-down strategy with bottom-up strategy
- *The system is view as having three layers*
  - A target layer in the middle
  - A layer above the target
  - A layer below the target
  - Testing converges at the target layer
- How do you select the target layer if there are more than 3 layers?
  - Heuristic: Try to minimize the number of stubs and drivers

[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

Integration Testing:
Sandwich Integration

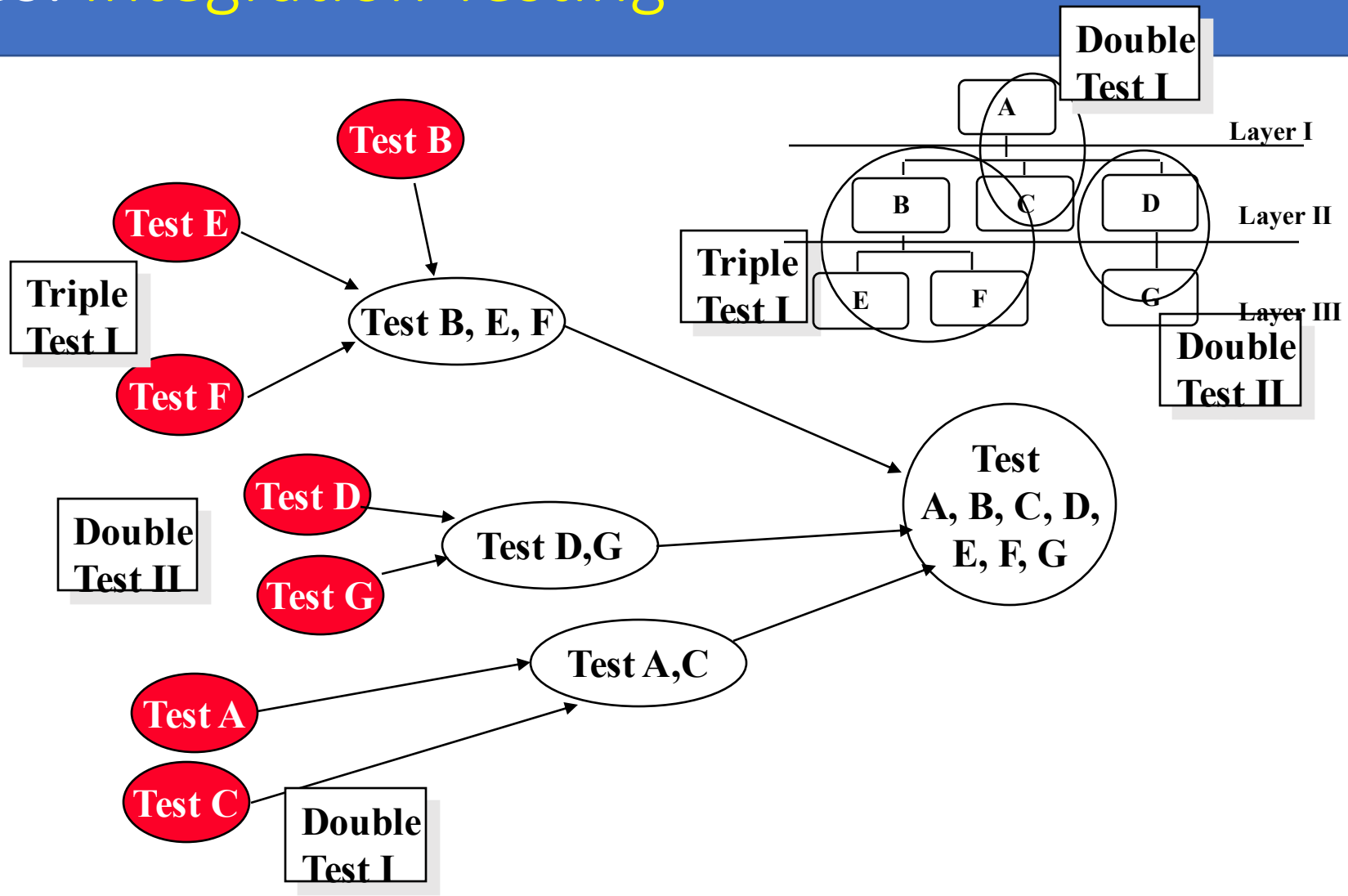




[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

Integration Testing:
Sandwich Integration



[Adapted from Pressman 4th Ed]

Testing Activities: Integration Testing

Steps in Integration-Testing

1. Based on the integration strategy, *select a component* to be tested. Unit test all the ***classes*** in the component.
2. Put selected ***component*** together; do any *preliminary fix-up* necessary to make the integration test operational (drivers, stubs)
3. Do *functional testing:* Define test cases that exercise all ***uses cases*** with the selected component
4. Do *structural testing:* Define test cases that exercise the selected ***component***
5. Execute *performance tests*
6. *Keep records* of the test cases and testing activities.
7. Repeat steps 1 to 7 until the full system is tested.

The primary *goal of integration testing is to identify errors* in the (current) component configuration.

[Adapted from Pressman 4th Ed]

Testing Activities: System Testing

- Functional Testing
- Structure Testing
- Performance Testing
- Acceptance Testing
- Installation Testing

Impact of requirements on system testing:

- The more explicit the *requirements*, the easier they are to test.
- Quality of *use cases* determines the ease of functional testing
- Quality of subsystem decomposition determines the ease of structure testing
- Quality of *nonfunctional requirements and constraints* determines the ease of performance tests:

[Adapted from Pressman 4th Ed]

Testing Activities: System Testing

Structure Testing

- *Essentially the same as white box testing.*
- Goal: Cover all paths in the system design
  - Exercise all input and output parameters of each component.

What's a a component again?

  - Exercise all components and all calls (each component is called at least once and every component is called by all possible callers.)

How do we know about all possible callers?

  - Use conditional and iteration testing as in unit testing.

[Adapted from Pressman 4th Ed]

Testing Activities: System Testing

Functional Testing

Essentially the same as black box testing

- Goal: Test functionality of system
- **Test cases are designed from the requirements analysis document (better: user manual) and centered around requirements and key functions (*use cases*)**

What's in a use case?
What do we test

- The system is treated as black box.
- Unit test cases can be reused, but in end user oriented new test cases have to be developed as well.

[Adapted from Pressman 4th Ed]

Testing Activities: System Testing

Performance Testing

- Stress Testing
  - Stress limits of system (maximum # of users, peak demands, extended operation)
- Volume testing
  - Test what happens if large amounts of data are handled
- Configuration testing
  - Test the various software and hardware configurations
- Compatibility test
  - Test backward compatibility with existing systems
- Security testing
  - Try to violate security requirements
- Timing testing
  - Evaluate response times and time to perform a function
- Environmental test
  - Test tolerances for heat, humidity, motion, portability
- Quality testing
  - Test reliability, maintain- ability & availability of the system
- Recovery testing
  - Tests system's response to presence of errors or loss of data.
- Human factors testing
  - Tests user interface with user

Is this list enough? What are the 2 types of requirements?

[Adapted from Pressman 4th Ed]

Testing Activities: System Testing

Test cases for Performance Testing

- **Push the (integrated) system to its limits.**
- **Goal: Try to break the subsystem**
- **Test how the system behaves when overloaded.**
  - Can bottlenecks be identified? (First candidates for redesign in the next iteration
- **Try unusual orders of execution**
  - Call a receive() before send()
- **Check the system's response to large volumes of data**
  - If the system is supposed to handle 1000 items, try it with 1001 items.
- **What is the amount of time spent in different use cases?**
  - Are typical cases executed in a timely fashion?

How many abnormal/exceptional cases can there be?

[Adapted from Pressman 4th Ed]

Testing Activities: System Testing

Acceptance Testing

- Goal: Demonstrate system is ready for operational use
  - Choice of tests is made by *client*/sponsor
  - Many tests can be taken from integration testing
  - Acceptance test is performed by the client, not by the developer.
- Majority of all bugs in software is typically found by the client after the system is in use, not by the developers or testers. Therefore two kinds of additional tests:

- *Alpha test:*
  - Sponsor uses the software at the *developer's site.*
  - Software used in a controlled setting, with the developer always ready to fix bugs.
- *Beta test:*
  - Conducted at *sponsor's site* (developer is not present)
  - Software gets a realistic workout in target environ- ment
  - Potential customer might get discouraged

What's ethnomethodology?

[Adapted from Pressman 4th Ed]

Testing Activities: System Testing

Testing Life Cycle

Establish the test objectives

Design the test cases

Write the test cases

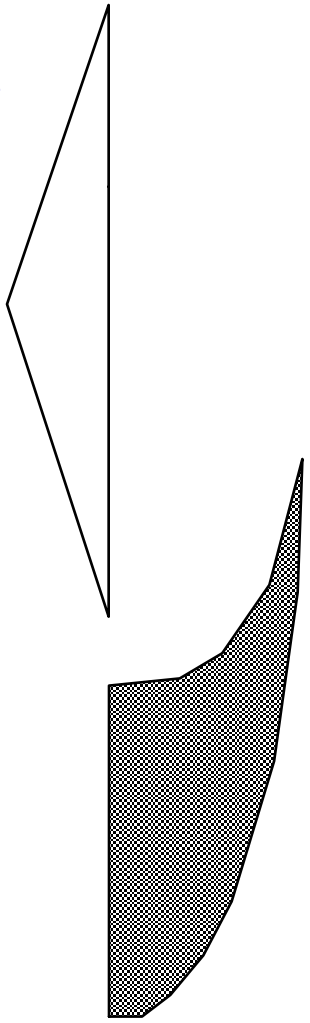
Test the test cases

Execute the tests

Evaluate the test results

Change the system

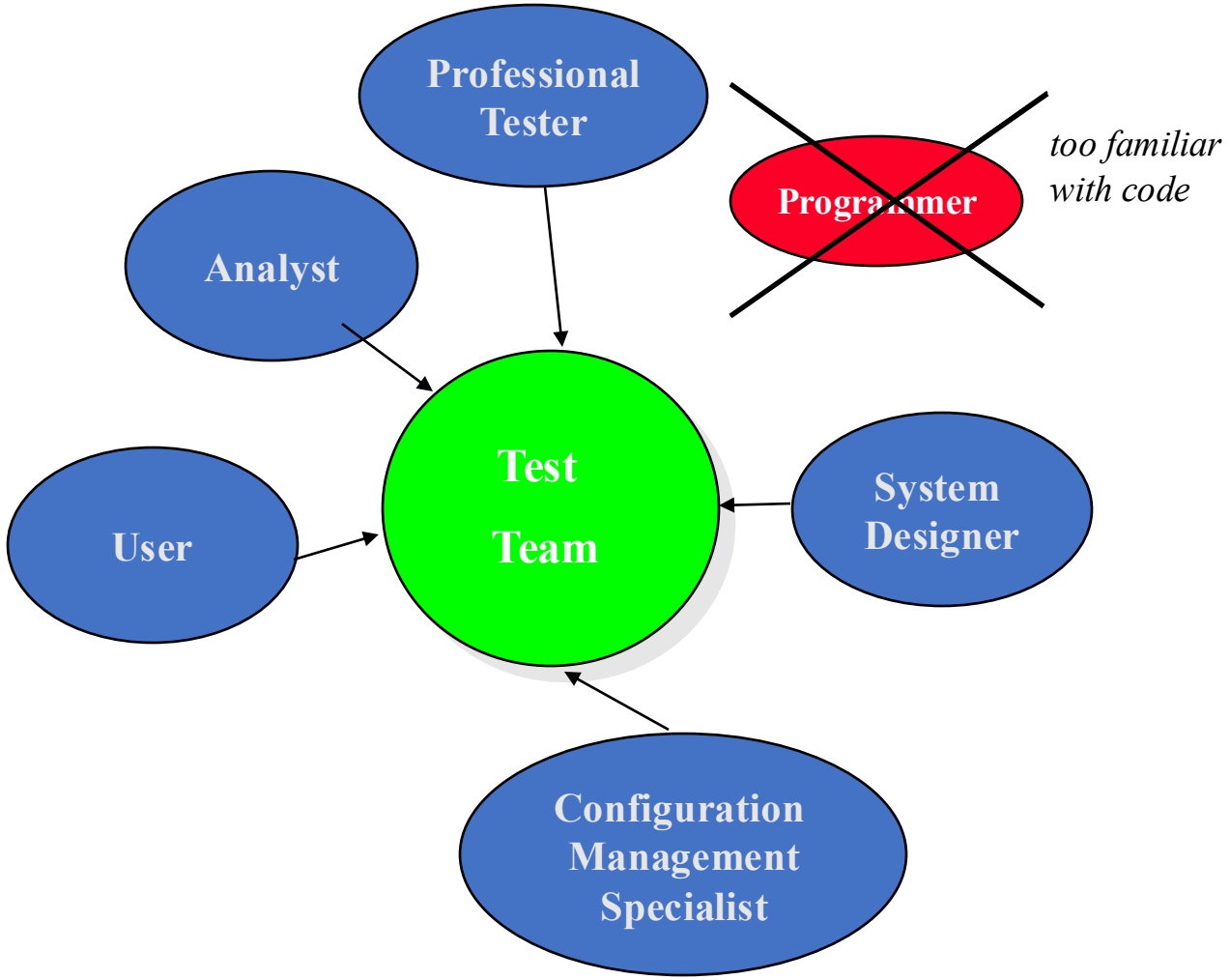
Do regression testing



[Adapted from Pressman 4th Ed]

Testing Activities: System Testing

Test Team

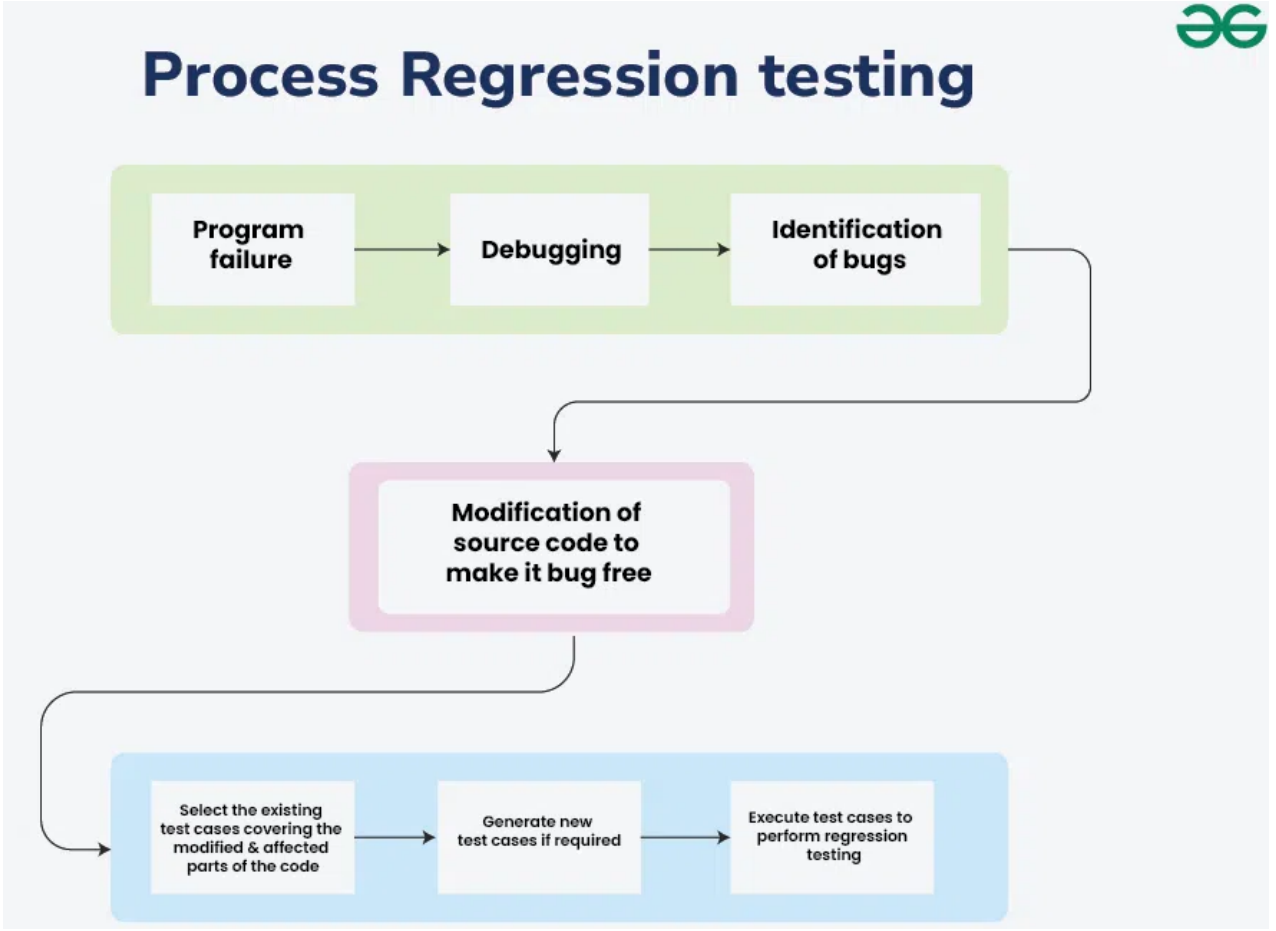


Albert Einstein - We can't solve problems by using the same kind of thinking we used when we created them.

[Adapted from Pressman 4th Ed]

Testing Activities: Regression Testing

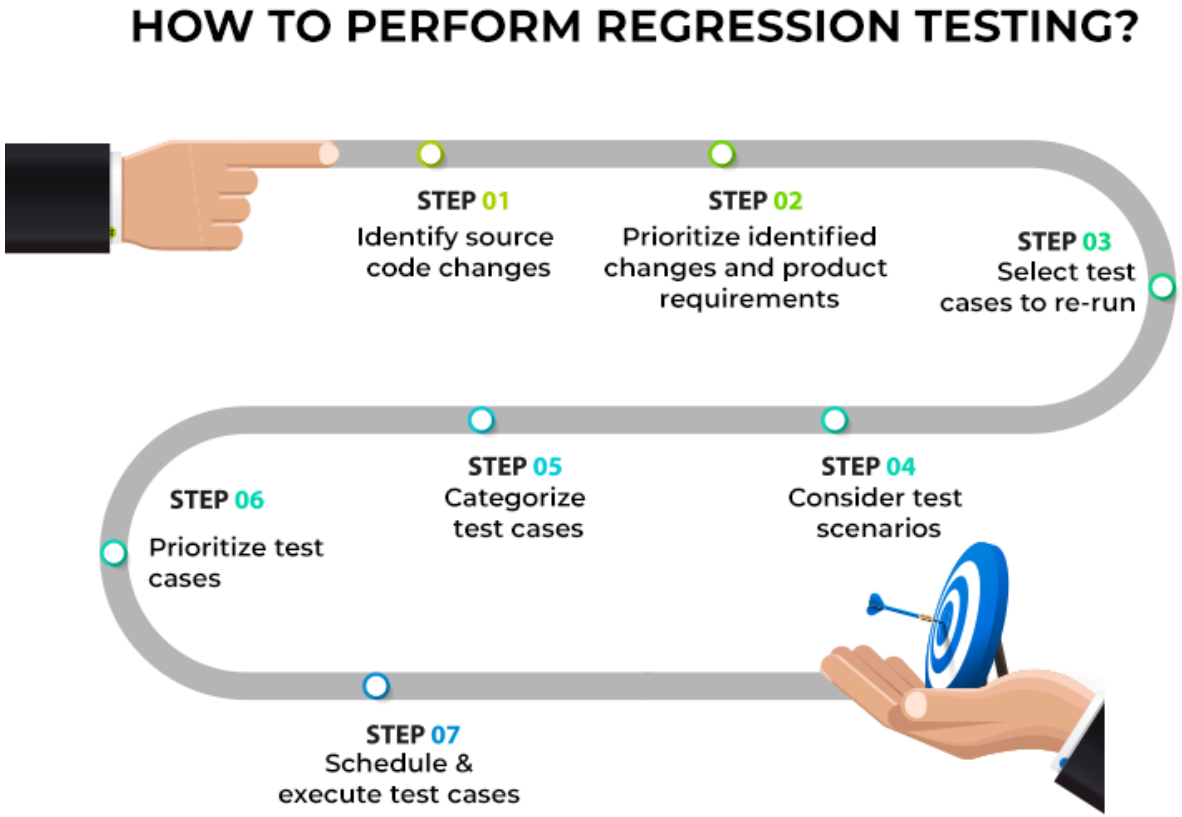
Regression testing *is defined as a type of software testing technique that re-runs functional and non-functional tests to ensure that a software application works as intended after any code changes, updates, revisions, improvements, or optimizations.*



[Adapted from Pressman 4th Ed]

Testing Activities: Regression Testing

- You should preserve all the test cases for a program
- During the maintenance phase, when a change is made to the program, the test cases that have been saved are used to do **regression testing**
  - figuring out if a change made to the program introduced any faults
- Regression testing is crucial during maintenance
  - It is a good idea to automate regression testing so that all test cases are run after each modification to the software
- When you find a bug in your program you should write a test case that exhibits the bug
  - Then using regression testing you can make sure that the old bugs do not reappear



[Adapted from Pressman 4th Ed]

Testing Activities: Regression Testing

Types of Regression Testing

- **Corrective Regression Testing** – Run existing test cases without modification when no major changes occur.
- **Retest-All Regression Testing** – Execute all previous test cases, which is time-consuming but thorough.
- **Selective Regression Testing** – Re-run only selected test cases based on the impact of code changes.
- **Progressive Regression Testing** – Used when modifying existing functionality and updating test cases accordingly.
- **Complete Regression Testing** – Performed after major updates to check system-wide stability.

[Adapted from Pressman 4th Ed]

Automated Testing Frameworks

- Introduction to Tools like Selenium and JUnit
- Integrating Testing Tools with CI/CD Pipelines

Automated Testing Framework

- Automated testing refers to the techniques which generate the test sets automatically
- The repeatability of test execution can be achieved with automation.
- There are multiple tools available for automated testing like
  - Selenium, JUnit
- Once a fault is corrected as a result of a failure, the test that uncovered the failure can be repeated to ensure that the failure does not occur anymore.
- Moreover, other tests can be run to ensure (to a limited extent) that no new faults have been introduced.
- Moreover, when tests are repeated many times, for example, in the case of refactoring , the cost of testing is decreased substantially.

[Adapted from Pressman 4th Ed]

Automated Testing Framework

- Automated testing refers to the techniques which generate the test sets automatically
- The repeatability of test execution can be achieved with automation.
- There are multiple tools available for automated testing like
  - Selenium, JUnit
- Once a fault is corrected as a result of a failure, the test that uncovered the failure can be repeated to ensure that the failure does not occur anymore.
- Moreover, other tests can be run to ensure (to a limited extent) that no new faults have been introduced.
- Moreover, when tests are repeated many times, for example, in the case of refactoring , the cost of testing is decreased substantially.

[Adapted from Pressman 4th Ed]

Automated Testing Framework

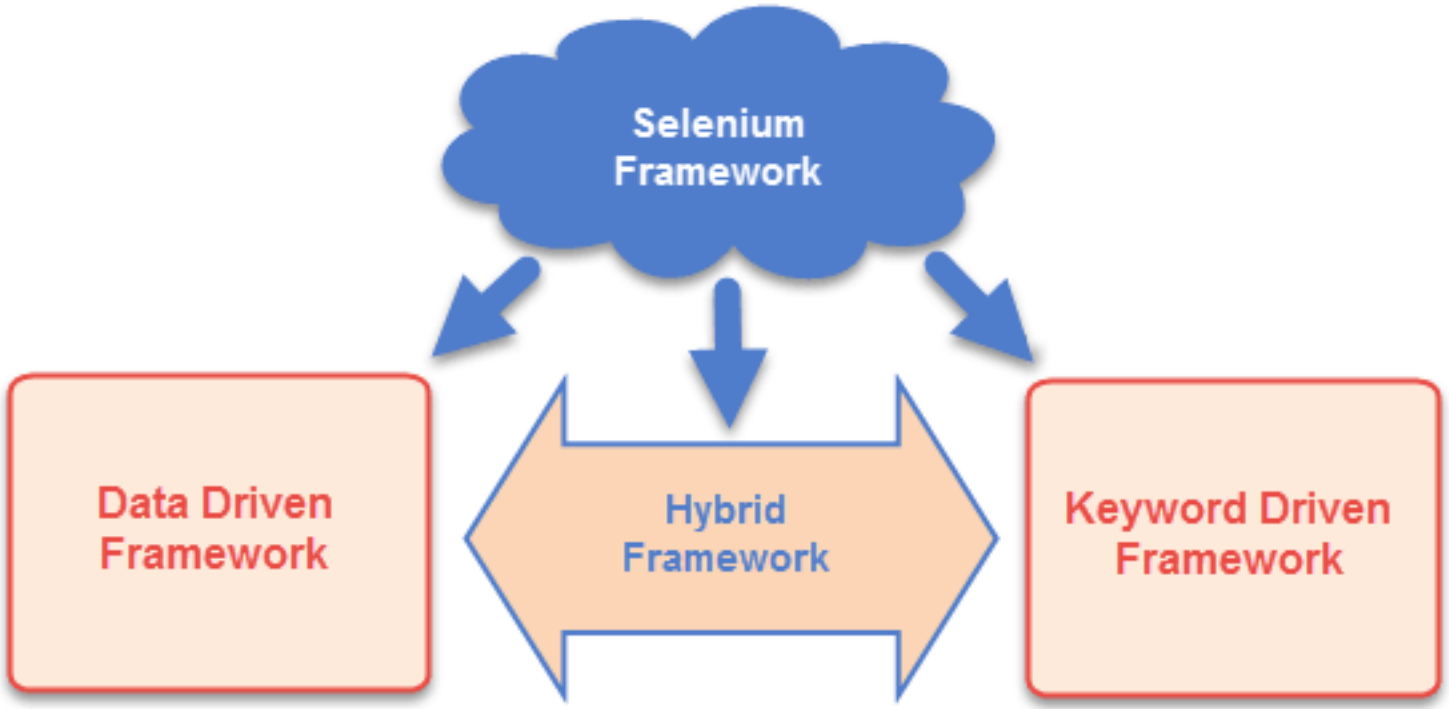
- Selenium Framework
- Selenium Framework is a suite of automation testing tools based on the JavaScript framework.
- It could run the tests directly on the target browser, drive the interactions on the required web page and rerun them without any manual input.
- It eliminates repetitive manual testing that consumes lots of time and effort.
- Conforms with the idea of Agile and DevOps, which endorse the continuous delivery workflow.



[Adapted from Pressman 4th Ed]

Automated Testing Framework

- Selenium Framework



[Adapted from Pressman 4th Ed]

Automated Testing Framework

- Junit Testing Framework

  - A framework for writing and automating the execution of unit tests for Java classes [JUnit, 2009].
  - The JUnit test framework is made out of a small number of tightly integrated classes
  - Developers write new test cases by subclassing the TestCase class.
  - The setUp() and tearDown() methods of the concrete test case initialize and clean up the testing environment, respectively.
  - The runTest() method includes the actual test code that exercises the class under test and compares the results with an expected condition.

[Adapted from Pressman 4th Ed]

Automated Testing Framework

- Junit Testing Framework

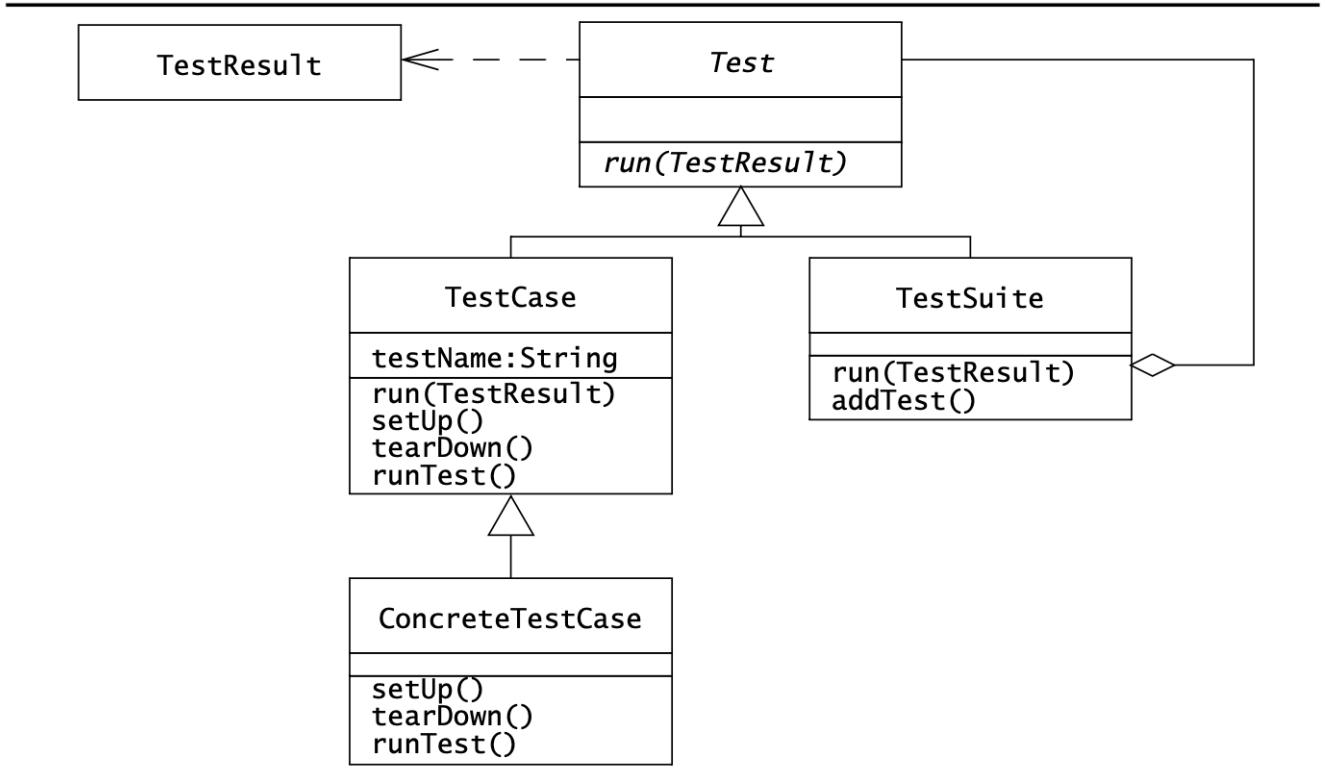


Figure 11-29 JUnit test framework (UML class diagram).

[Adapted from Pressman 4th Ed]

Automated Testing Framework

- Features of Junit Testing Framework

•Fixtures
  - •**Fixtures** is a fixed state of a set of objects used as a baseline for running tests. The purpose of a test fixture is to ensure that there is a well-known and fixed environment in which tests are run so that results are repeatable.

•Test suites
  - •A test suite bundles a few unit test cases and runs them together. In JUnit, both @RunWith and @Suite annotation are used to run the suite test.

•Test runners
  - •Test runner is used for executing the test cases.

•JUnit classes
  - •JUnit classes are important classes, used in writing and testing JUnits.

[Adapted from Pressman 4th Ed]

Automated Testing Framework

- Features of Junit Testing Framework

•**Junit Important Classes**

- •**Assert** – Contains a set of assert methods.
- •**TestCase** – Contains a test case that defines the fixture to run multiple tests.
- •**TestResult** – Contains methods to collect the results of executing a test case.

[Adapted from Pressman 4th Ed]

Automated Testing Framework

- **Elements of a CI/CD Pipeline**

A CI/CD pipeline tools consists of multiple interconnected phases, each fulfilling a distinct role in the software delivery journey. These stages typically include:

1) Version Control: Developers commit code changes to a version control system like Git.

2) Continuous Integration: The code changes are automatically integrated into a shared repository and tested using automated build processes.

3) Automated Testing: The code changes undergo various automated tests, including unit tests, integration tests, and end-to-end tests.

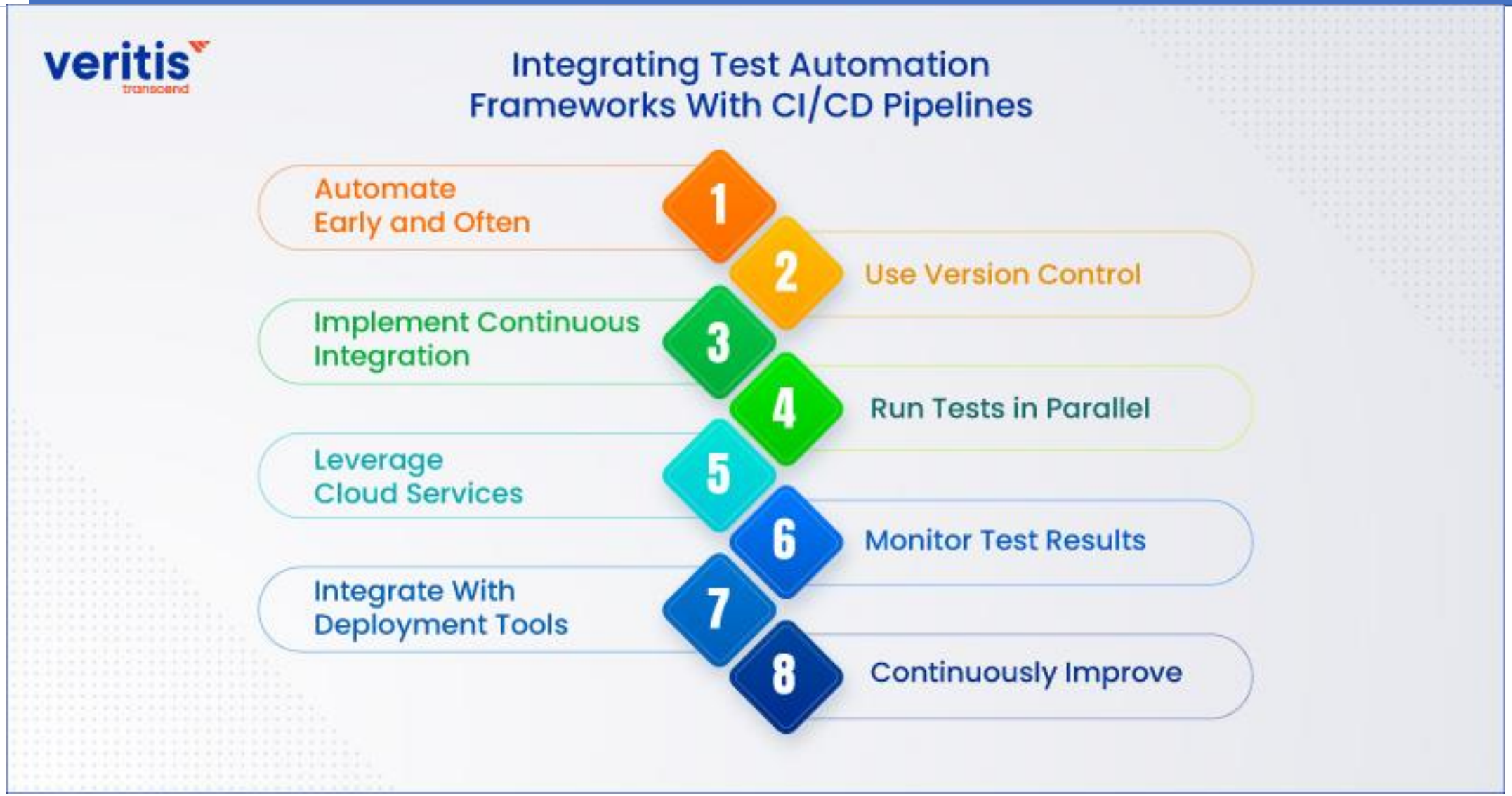
4) Artifact Generation: The build process generates deployable artifacts, such as binaries or Docker images.

5) Deployment: The artifacts are deployed to various environments, such as development, staging, and production.

6) Monitoring and Feedback: The deployed software is monitored in real-time, and feedback is provided to the development team.

[Adapted from Pressman 4th Ed]

Automated Testing Framework



[Adapted from Pressman 4th Ed]

Automated Testing Framework

1) Automate Early and Often

Start by automating your tests as early in the development cycle as possible. By incorporating automated tests into your CI/CD pipeline tools, you can detect and address issues early before they escalate into more significant problems.

2) Use Version Control

Ensure your test automation platform code resides alongside your application code within a version control system like ***Git***. This practice enables you to monitor modifications, collaborate effectively with team members, and revert to prior versions.

3) Implement Continuous Integration

Establish a continuous integration testing server, like ***Jenkins*** or ***Travis CI***, to automatically construct and evaluate your application whenever modifications are pushed to the repository. Utilize build scripts or configuration files to delineate your build and test procedures, ensuring reproducibility and uniformity.

[Adapted from Pressman 4th Ed]

Automated Testing Framework

4) Run Tests in Parallel

To speed up test execution, consider running tests in parallel across multiple environments or test machines. This allows you to complete testing faster and get feedback faster, enabling faster iterations and shorter release cycles.

5) Leverage Cloud Services

Use cloud-based testing services like ***Sauce Labs*** or ***BrowserStack*** to run tests on various browsers and devices. These services provide scalable infrastructure, enabling you to execute tests in parallel across multiple configurations effortlessly.

6) Monitor Test Results

Implement automated monitoring and reporting mechanisms to track the results of your tests over time. Use dashboards, alerts, and notifications to keep stakeholders informed about the status of your tests and any issues that arise.

[Adapted from Pressman 4th Ed]

Automated Testing Framework

7) Integrate With Deployment Tools

Integrate your test automation framework with deployment tools like Ansible, Chef, or Puppet to automate the deployment of your application. This ensures that your tests are always executed against the latest version of your software and that deployment-related issues are detected early.

8) Continuously Improve

Regularly review and refactor your test automation platform code to remain maintainable, reliable, and efficient. Look for opportunities to optimize test execution, increase test coverage, and reduce test flakiness.

[Adapted from Pressman 4th Ed]